



Explore | Expand | Enrich

<Codemithra />TM

STACK PERMUTATION



STACK PERMUTATION

EXPLANATION

A stack can be used to generate permutations of a sequence.

- Given a sequence and a target permutation, we can check if the target permutation is achievable using a stack by simulating the stack operations.
- The idea is to start with an empty stack and sequentially push and pop elements from the original sequence until the stack matches the target permutation.

STACK PERMUTATION

EXPLANATION

Let array $a[]$: 1, 2, 3 and array $b[]$: 2, 1, 3

Steps:

push 1 from array $a[]$ to stack

push 2 from array $a[]$ to stack

pop 2 from stack to array $b[]$

pop 1 from stack to array $b[]$

push 3 from array $a[]$ to stack

pop 3 from stack to array $b[]$

Therefore, Output : Yes

STACK PERMUTATION

ALGORITHM

- Create an empty stack to simulate the stack operations.
- Iterate through the original sequence and perform the following steps:
 - Push the current element onto the stack.
 - Check if the top element of the stack matches the next element in the target permutation.
- If they match, pop elements from the stack until they don't match or the stack is empty.
- After processing all elements in the original sequence, if the stack is empty, it means the target permutation is achievable; otherwise,



STACK PERMUTATION

PSEUDOCODE

```
function isStackPermutation(original, target):  
    stack = empty stack  
    i = 0  
    for element in original:  
        stack.push(element)  
        while (!stack.isEmpty() && stack.peek() == target[i]):  
            stack.pop()  
            i++  
    return stack.isEmpty()
```

STACK PERMUTATION

EXPLANATION

Input:

Original sequence: [1, 2, 3]

Target permutation: [2, 1, 3]

Output:

Is it a stack permutation? true

STACK PERMUTATION

EXPLANATION

- The original sequence [1, 2, 3] can be transformed into the target permutation [2, 1, 3] using stack operations.
- We push elements from the original sequence onto the stack and then check if they match the elements in the target permutation.
- In this case, the stack operations can produce the target permutation, so the output is "true."



STACK PERMUTATION

```
import java.util.Stack;

public class Main {
    public static boolean
isStackPermutation(int[] original, int[]
target) {
    Stack<Integer> stack = new
Stack<>();
    int i = 0;

    for (int element : original) {
        stack.push(element);
        while (!stack.isEmpty() &&
stack.peek() == target[i]) {
            stack.pop();
            i++;
        }
    }
}
```

```
return stack.isEmpty();
    }

    public static void main(String[]
args) {
        int[] original = {1, 2, 3};
        int[] target = {2, 1, 3};

        boolean result =
isStackPermutation(original, target);
        System.out.println("Is it a
stack permutation? " + result);
    }
}
```

STACK PERMUTATION

TIME AND SPACE COMPLEXITY :

Time Complexity: $O(n)$, where n is the length of the original sequence.

Space Complexity: $O(n)$, as the stack can store up to n elements in the worst case.

INTERVIEW QUESTIONS

1. What is a stack permutation in the context of data structures?

Answer: A stack permutation is a permutation of a sequence that can be obtained using stack operations such as push and pop.

INTERVIEW QUESTIONS

2. How do you determine if a given permutation is a stack permutation?

Answer: To check if a permutation is a stack permutation, we simulate the stack operations on the original sequence and see if it matches the target permutation.

INTERVIEW QUESTIONS

3. What is the key algorithmic idea behind identifying a stack permutation?

Answer: The key idea is to start with an empty stack, push elements from the original sequence onto it, and pop elements if they match the next element in the target permutation.

INTERVIEW QUESTIONS

4. What is the time complexity of checking a stack permutation?

Answer: The time complexity is $O(n)$, where n is the length of the original sequence.

INTERVIEW QUESTIONS

5. What is the space complexity of checking a stack permutation?

Answer: The space complexity is $O(n)$ because the stack can store up to n elements.

INTERVIEW QUESTIONS

6. Can you give an example of a valid stack permutation?

Answer: For example, the target permutation $[3, 1, 2]$ can be achieved from the original sequence $[1, 2, 3]$ using stack operations.



<https://learn.codemithra.com>



THANK YOU



+91 78150 95095



codemithra@ethnus.com



www.codemithra.com